RBE201TC

RDK Robot Laboratory Manual

Computer Aided Design and Simulation with Sustainability Thinking in Robotics

Module Leader: Dr. Zhouyan Qiu

Learning Outcomes

A. Apply sustainability principles to real challenges in robotics.

B. Use MATLAB to simulate robotic systems and analyse behaviour.

C. Write MATLAB code to analyse and visualise robot data.

D. Use SolidWorks to design and manufacture components for sustainable robotic applications.

Manual Purpose and Baseline Requirements

An indoor robot autonomously follows a predefined black-tape route to collect temperature and relative humidity measurements at locations throughout the accessible area of the room.

The ultimate goal is to generate indoor temperature and relative humidity maps and analyse their spatial patterns.

The project integrates mechanical design, sensing, control, software and hardware to achieve this goal.

CAD models, simulations, the physical robot and measured data provide consistent evidence supporting the design, mapping results and analysis.

Baseline Requirements

- Assemble and verify the robot for autonomous route following.
- Collect timestamped temperature and relative humidity measurements linked to locations across the accessible room.
- Use MATLAB to generate indoor temperature and relative humidity maps and analyse spatial patterns, sampling coverage and limitations.

Extensions are grouped into five areas:

- **Vehicle Design:** Improve the vehicle’s appearance, component layout, and overall build quality.
- **Vehicle Sustainability:** Improve material use, structural performance, or energy efficiency through lightweight design, motion/FEA analysis, or validated energy modelling.
- **Line Following and Navigation:** Extend navigation with camera-based line tracking, obstacle avoidance, or improved robustness.
- **Temperature and Humidity Data Collection:** Improve measurement quality, sampling coverage, data logging, or real-time transmission via a local web interface or MQTT.
- **Temperature and Humidity Mapping:** Enhance the final maps with a room floor plan, spatial heatmaps, dew point, or discomfort index.

Five-person teams must implement at least two distinct functional extensions, which may fall within the same area. Multiple extensions within an area are permitted, but marks for each area are capped. Achieving high marks therefore requires extensions across multiple areas.

Provided Hardware

- RDK X5
- motor driver
- four encoder motors
- four-channel tracking module
- DHT11 temperature/RH module
- Battery Pack: two 12V batteries with adapter
- voltage regulator module
- Camera

Custom Components

Design in SolidWorks and 3D print: Chassis parts, connectors

1 Prepare the RDK Board and Chassis

1. Identify the RDK board and insert the supplied microSD card into the slot on the underside in the orientation shown.

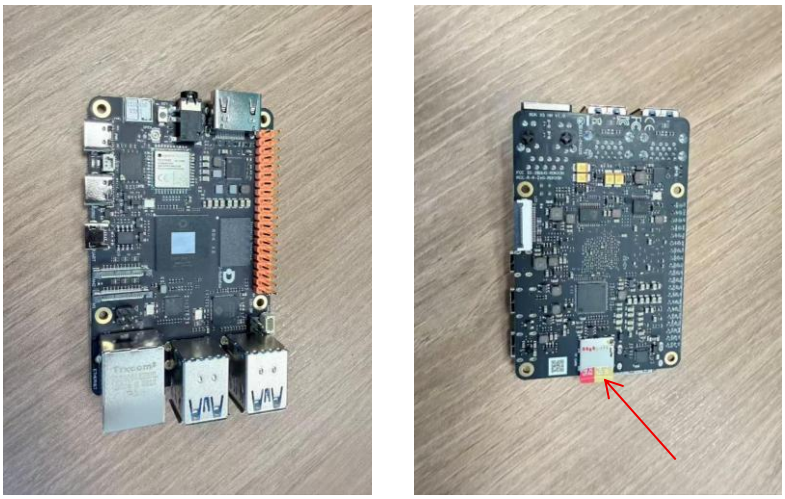


Figure 1 RDK board and microSD card slot on the underside.

2. Install the four motors and wheels on the chassis. Secure the RDK, motor driver, batteries and sensors, leaving access to switches and connectors. Conductive fasteners must not touch exposed circuitry.

2 Connect the Line Sensors and DHT11

Disconnect power before wiring. Follow the annotated diagram to make the six connections for the line sensor module and the three connections for the DHT11. Check the board orientation and the labels on the actual modules; do not identify signals by wire colour alone.

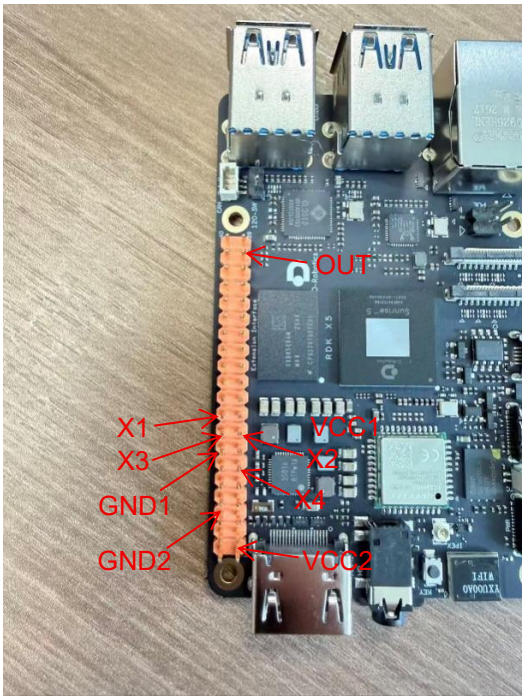


Figure 2 Annotated GPIO wiring diagram. Keep the actual board in the same orientation when comparing it with the diagram.

Module	Connections	Checks
Line sensor module	VCC1, GND1, X1, X2, X3, X4	Check all six connections against the diagram.
DHT11 module	VCC2, GND2, OUT	Check all three connections. OUT must match the source code configuration.

Module	Connections	Checks
RDK GPIO	Use the pinout for the actual board.	Confirm logic compatibility. GPIO uses 3.3 V logic.

The signal names in Figure 2 are not physical pin numbers. Before switching on, check both the pinout of the actual board and the software configuration. If the hardware revision differs, do not infer the GPIO mapping from photographs alone.

Checkpoint Before switching on, check every connection with a teammate or instructor and record the pin mapping used by your group.

3 Inspect the Sensor Wiring

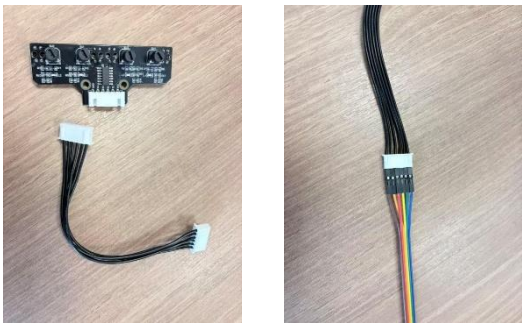


Figure 3 Line sensor module wiring harness and jumper connections.

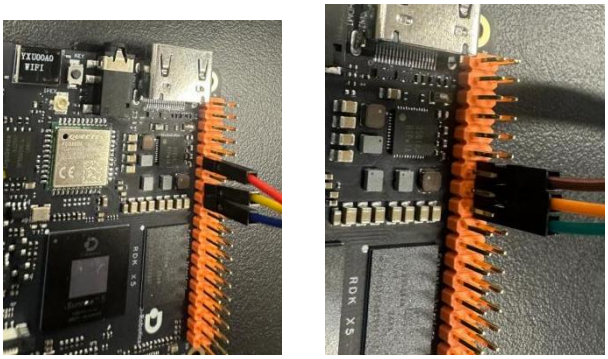


Figure 4 Line sensor connections at the RDK end.

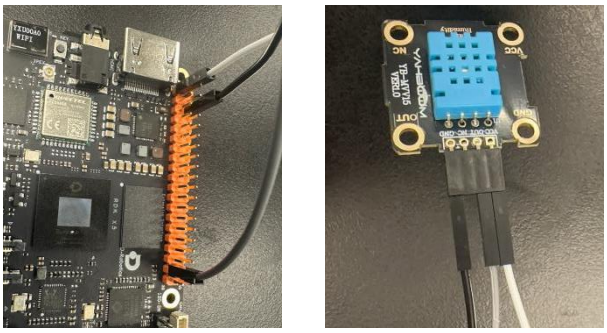


Figure 5 DHT11 connections at the board end and the sensor itself.

1. Trace VCC and GND from each module to the specified pins. Check for connectors shifted by one pin.

2. Check that each line sensor output, X1–X4, connects to the input expected by the source program. Identify the left and right sensor positions relative to the direction of forward travel.

3. Secure the wires without pulling on the pins. Expose the DHT11 to room air and keep it at the same height throughout the room measurements.

Checkpoint Use the photographs to support the inspection. Confirm operation with the channel tests in Section 10; visual similarity alone does not establish that the wiring works.

4 Connect the Motor Driver

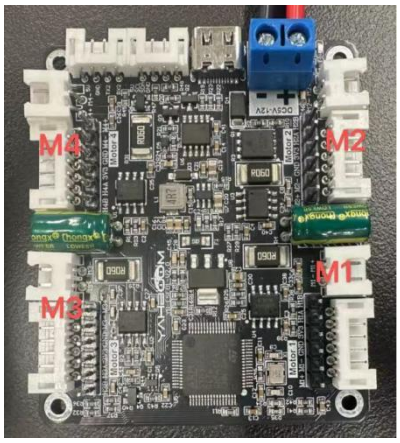


Figure 6 M1–M4 channel labels on the example motor driver.

1. Connect each motor and its encoder wires to the corresponding channel. Record which wheel is connected to M1–M4.

2. Connect the motor driver to the RDK using the supplied USB data cable. Keep the motor power supply path separate from the regulated RDK power supply path.

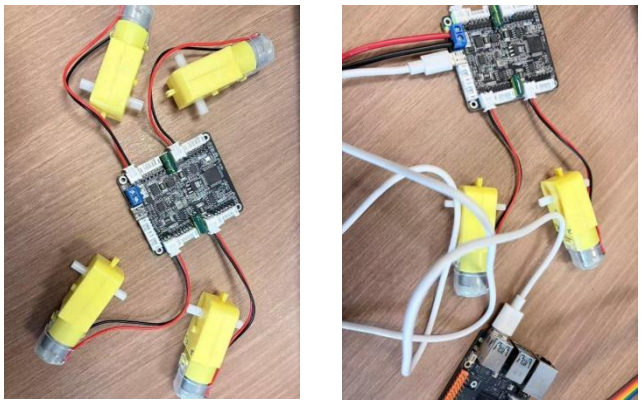


Figure 7 Four motor connections and the connection between the motor driver and RDK.

Channel	Wheel position	Forward rotation verified
M1		
M2		
M3		
M4		

Checkpoint All four sets of connections are secure. Verify channel order and rotation direction with the wheels raised before placing the robot on the floor.

5 Connect and Check the Power Supplies

The illustrated setup uses two separate batteries: one powers the RDK through a voltage regulator, and the other powers the motor driver. The RDK X5 USB Type-C power input requires a suitable 5 V supply rated at 5 A. Check the actual board and voltage regulator before connecting them.

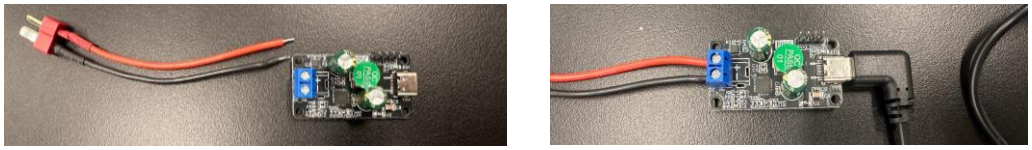


Figure 8 Voltage regulator input wiring and output connections in the setup.

1. Identify INPUT and OUTPUT from the component labels. Keep the RDK disconnected and use a multimeter to check the regulated output voltage and polarity. Red indicates positive and black indicates negative in the illustration; verify the actual wiring before use.

2. Loosen the terminal screws, fully insert the correct wires and tighten the screws. Ensure that no loose copper strands or exposed conductors bridge adjacent terminals.

3. Connect the verified regulated output to the RDK power input. Never apply the battery voltage directly. The regulator, wires and connectors must meet the current requirements.

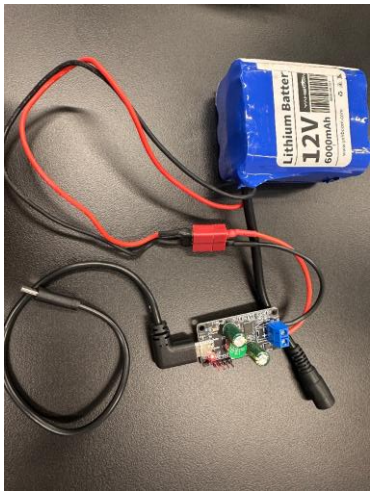


Figure 9 Example battery and voltage regulator.

4. Check the rated voltage range and polarity of the motor driver before connecting the other battery. Use an approved charger compatible with the supplied batteries.

6 Connect the Display and Start the RDK

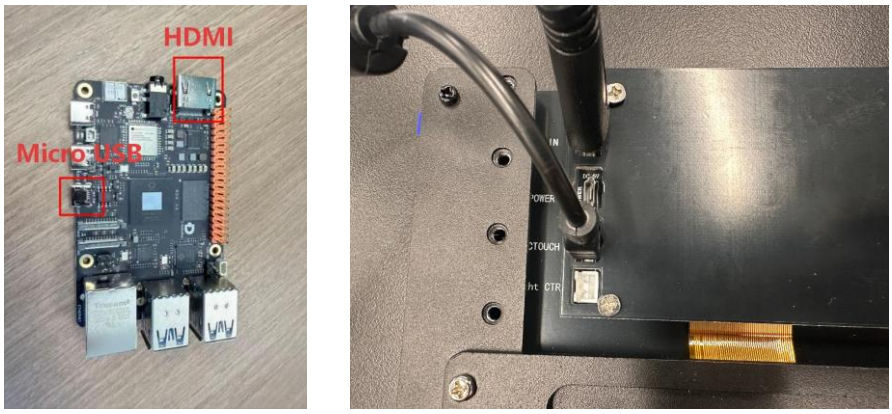


Figure 10 HDMI location and display connector. The Micro USB connector in the board photograph is a debug interface, not the RDK power input.

1. Before powering the display, connect the HDMI cable between the RDK and display. Use an adapter compatible with the supplied screen.

2. Connect the power cable specified for the screen. The illustrated kit uses the CTOUCH or POWER connector as directed by the screen instructions. Check the labels and power requirements of the actual screen.

3. Connect the keyboard and mouse to USB host ports, switch on the verified RDK power supply, and wait for the desktop or login screen.

4. Connect to the current laboratory Wi-Fi through the desktop network menu. Press Ctrl+Alt+T to open a terminal.

Record Startup Information

```
hostname
cat /etc/os-release
python3 --version
hostname -I
```

Record the hostname, operating system version and reachable IP address. The board may have multiple network interfaces. Identify the current laboratory network rather than assuming the first address is correct.

Checkpoint　The RDK starts reliably, and the keyboard and terminal work. Resolve repeated restarts or unstable power before running any motor program.

7 Prepare the Software Environment

Run the following commands on the RDK using the image and code package. Installing software requires internet access. Avoid a full system upgrade or replacing the system image during the laboratory session.

```
sudo apt update
sudo apt install -y gcc g++ make util-linux git \
  python3 python3-pip python3-dev python3-serial
python3 -c "import serial; print(serial.__version__)"
python3 -c "import Hobot.GPIO; print('Hobot.GPIO OK')"
sudo python3 -c "import serial; import Hobot.GPIO; print('Dependencies OK')"
command -v gcc chrt taskset
```

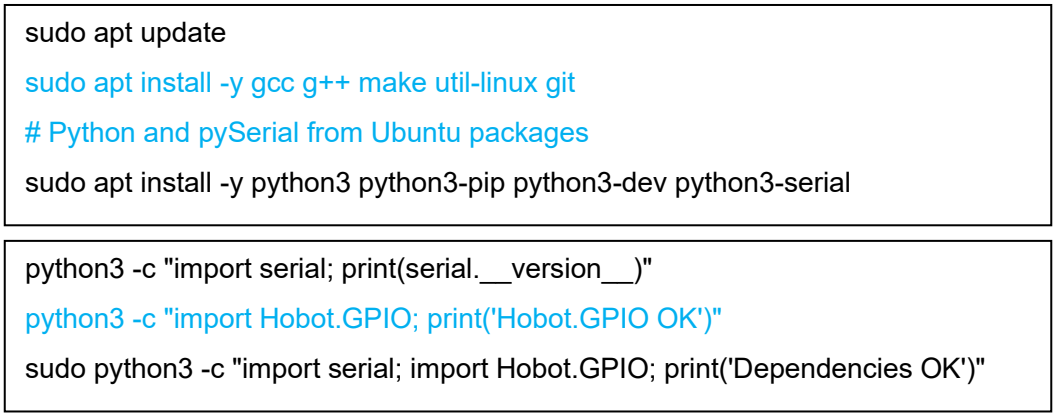


Figure 11　Package installation and Python dependency checks.

The package python3-serial provides pySerial, imported as serial. Hobot.GPIO must be available in the supplied RDK image. Expect a version number, Hobot.GPIO OK and Dependencies OK from the three Python checks; command -v must print paths for gcc, chrt and taskset. If a check fails, resolve the image or Python environment before continuing. The integrated application runs under sudo, so its dependencies must also import under sudo python3.

Configure Serial Port Permissions

```
sudo usermod -aG dialout "$USER"
```

Log out and log back in, or restart the system when safe, to apply the group membership change. Then check:

```
groups
ls -l /dev/ttyUSB* /dev/ttyACM*
```

Only existing devices will be listed; an unmatched device pattern may produce a not-found message. Identify the actual serial port of the motor driver and ensure that it matches the configuration. Do not run two programs that control the same serial port at the same time.

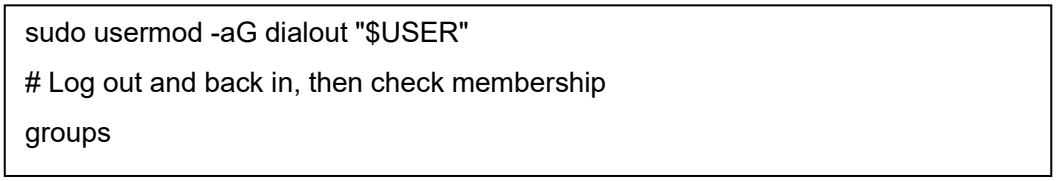


Figure 12　Apply serial-port group membership before testing.

Obtain and Locate the Source Files

Copy the supplied code package to the RDK. Create the working directory below and put all five source files directly inside it; avoid a nested RBE201TC/RBE201TC directory. Keep the two C files beside the three Python files.

```
mkdir -p ~/RBE201TC
cd ~/RBE201TC
pwd
ls
```

moto_test.py — Motor test
line_sensor_test.py — Line sensor input
dht11_rdk_x5_test.c — Temperature and humidity test
dht11_rdk_x5_gpio401.c — Temperature and humidity helper program
line_follower.py — Integrated application

Software defaults for this package are /dev/ttyUSB0 at 115200 baud; tracking inputs X1/X2/X3/X4 on physical header pins 15/18/16/11 using GPIO.BOARD; and DHT11 data on Linux GPIO 401, identified in the C files as physical pin 37. Check these against your wiring. The README repeats the package-specific settings; the build and running instructions are given in Sections 10 and 11.

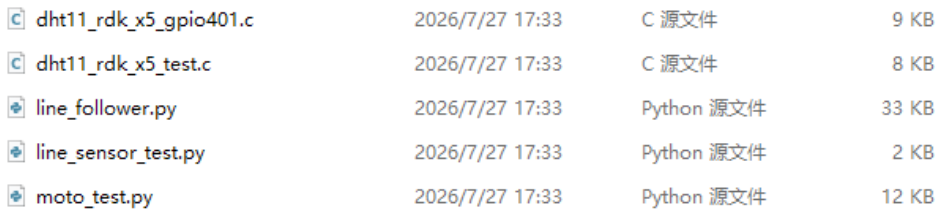


Figure 13 Folder content overview.

8 Configure Networking and SSH

Connect Using the Current IP Address

On the RDK, identify the active network profile and IP address, then connect to that address from your group computer. The classroom username and password for this kit have been confirmed.

```
hostname -I
nmcli connection show --active

ssh sunrise@10.40.94.4
# Example address only; replace with the RDK address.
# Password: sunrise
```

In your editor's SSH configuration, choose any name for Host. Set HostName to the current RDK address and User to sunrise. Port is normally 22.



Figure 14 Three IP addresses in the endpoint.

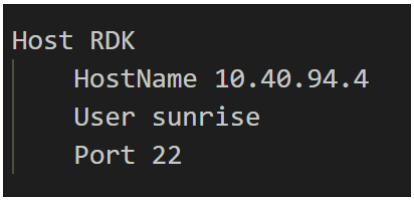


Figure 15 SSH Connection Configuration File.

Use the Static Address Assigned to Your Group

Confirm the current laboratory connection name, subnet, gateway, DNS servers and assigned address with the instructor. Do not apply these settings unchanged to another network.

```
sudo nmcli connection modify "H3CWIFI03" \
  ipv4.method manual \
  ipv4.addresses "10.40.94.204/24" \
  ipv4.gateway "10.40.94.1" \
  ipv4.dns "10.40.94.1 8.8.8.8"
sudo nmcli connection up "H3CWIFI03"
```

In the classroom addressing scheme, Group 4 uses .204 and Group 13 uses .213. Use only the address assigned to your group. Activating a new address may disconnect SSH. Reconnect using the new address and retain access through a local keyboard and display.

Verify and Record the Configuration

```
hostname -I
nmcli connection show --active
```

Connection name / SSID _____
Address and subnet _____
Gateway and DNS servers _____
SSH reconnection verified _____

Checkpoint The address is used only by your group, the RDK is reachable, and the recorded settings match the current laboratory network.

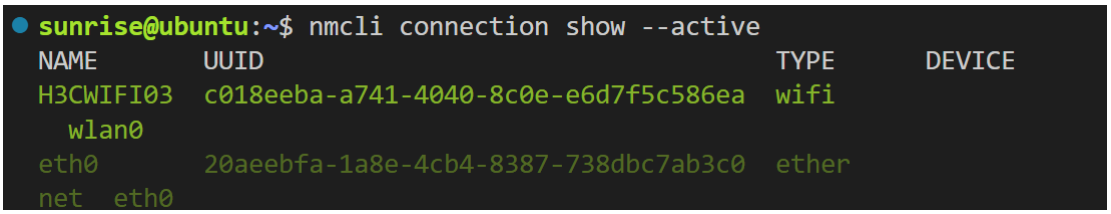


Figure 16 The terminal will display the WiFi name.

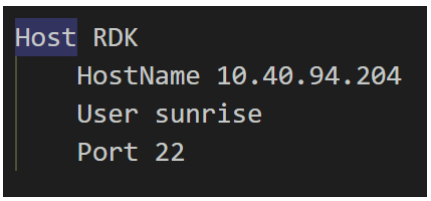


Figure 17 Second configuration update.

9 Configure ROS 2 and Measurement Time

ROS 2 Domains for Multiple Nodes

When using ROS 2, load the environment supplied with the image and use the same assigned domain ID on your group's RDK and all group computers. Different groups use different IDs to separate ROS discovery.

```
printenv ROS_DISTRO
ls /opt/ros
# Load the setup.bash for the installed distribution.
nano ~/.bashrc
```

```
  GNU nano 6.2                  /home/sunrise/.bashrc *
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "**" used in a pathname expansion context will
# match all files and zero or more directories and subdirectories.
#shopt -s globstar
```

Figure 18　Open nano as shown in the figure.

Add or update a single domain setting in .bashrc. The following example is for Group 4:

```
export ROS_DOMAIN_ID=4
```

```
sunrise@ubuntu:~$ source ~/.bashrc
echo $ROS_DOMAIN_ID
4
```

Figure 19　Terminal output is shown in the figure.

In nano, press Ctrl+O, Enter and Ctrl+X in sequence. Open a new terminal or reload the file. Restart the ROS command-line daemon if it was previously started with different settings.

```
source ~/.bashrc
echo "$ROS_DOMAIN_ID"
ros2 daemon stop
ros2 daemon start
```

```
sunrise@ubuntu:~$ ros2 daemon stop; ros2 daemon start
The daemon is not running
The daemon has been started
```

Figure 20　Upon successful completion, the terminal displays the result as shown in the figure.

If ros2 is not found, check the environment first. Setting this variable does not turn standalone serial test programs into ROS nodes.

Use a Reliable Measurement Clock

Before measuring the room, compare the RDK clock with the laboratory time reference and record the time zone and synchronisation method. Use network time synchronisation when available. If internet access is unavailable, manually set the actual current time before data collection begins.

```
date -Is
# Replace the example below with the actual date and time.
sudo date -s "2026-09-06 14:00:00"
date -Is
```

```
sunrise@ubuntu:~$ sudo date -s '2026-07-30 16:40:00'
date
2026年 07月 30日 星期四 16:40:00 CST
2026年 07月 30日 星期四 16:40:00 CST
```

Figure 21　Terminal output is shown in the figure.

The final date command only displays the time; it does not synchronise the clock. Do not change the clock during a measurement run. Record any clock jumps or corrections before analysing timestamps.

Checkpoint　Devices using ROS have the correct group domain ID. The measurement clock and time zone are recorded before data collection.

10 Bench Test the Motors and Line Sensors

Test A Motor Communication and Rotation

Secure the robot on a stable stand with all wheels clear of the surface. Ensure that the motor power can be disconnected easily. Run only one motor control program at a time.

```
cd ~/RBE201TC
python3 moto_test.py
```

```
# Enter the directory containing the source files
cd ~/RBE201TC
# Run the motor test with the wheels raised
python3 moto_test.py
```

Figure 22　The terminal is shown in the figure.

```
sunrise@ubuntu:~/Desktop/RBE201TC$ python3 moto_test.py
================================================
Yahboom Four-Channel Encoder Motor Driver Control Program
Press Ctrl+C at any time to stop the motors
================================================
✔ Successfully connected to serial port /dev/ttyUSB0

Select a test mode:
  1. Basic functionality test (all motors)
  2. Individual motor test
  3. Motor pair test (car mode)
  4. Manual control mode

Enter an option (1-4): 1

================================================
Starting basic functionality test
================================================

--- Configuring motor type: 1 ---
  Sent: $mtype:1#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,3,2#
  Received: Set mtype:OK!

>>> All motors rotate forward at 50% speed for 3 seconds

--- Setting speed: [ 500,  500,  500,  500] ---
  Sent: $spd:500,500,500,500#
  Received: ,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,3,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,2,2#
  Received: $MAll:26,-2,3,2#
```

Figure 23　The successful terminal display is shown in the figure.

1. Confirm that the program can open the configured serial port. If a menu appears, select the appropriate test as described in the source code package.

2. Observe all four motors and check the M1–M4 mapping. Record the rotation directions and any stalled or reversed motors. Stop the test before changing wiring or configuration.

3. Check how the program stops. First press Ctrl+C to end it. If any motion continues, disconnect motor power. Confirm a reliable stopping procedure before running the robot on the floor.

Test B Line Sensor Channel Response

```
cd ~/RBE201TC
sudo python3 line_sensor_test.py
```

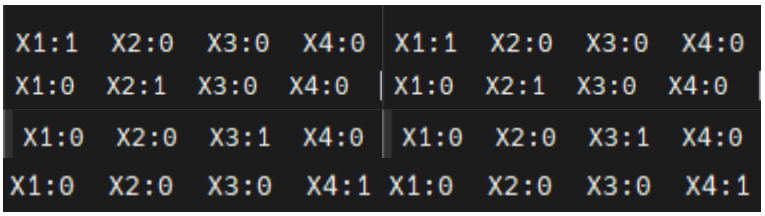


Figure 24 Data Example.

Place X1, X2, X3 and X4 over black tape and a light-coloured surface in turn. Record their digital states using the lighting and sensor height intended for the actual route. Changing inputs confirm a communication response, but do not yet verify steering control.

Channel	Light surface state	Black tape state	Physical position
X1			
X2			
X3			
X4			

Compare the measured signal polarity and left-to-right sensor order with the line-following program. If a channel never changes, first check alignment, wiring, sensor height and input configuration.

Checkpoint All motors respond as expected, the stopping procedure works, and each line sensor channel has a usable, recorded response.

11 Integrate Temperature and Humidity Logging

Compile the C Helper Programs

```
cd ~/RBE201TC
gcc dht11_rdk_x5_test.c -o dht11_test -Wall
gcc dht11_rdk_x5_gpio401.c \
  -o dht11_rdk_x5_gpio401_server -Wall
chmod +x dht11_rdk_x5_gpio401_server
```

```
gcc dht11_rdk_x5_test.c -o dht11_test -Wall
gcc dht11_rdk_x5_gpio401.c -o dht11_rdk_x5_gpio401_server -Wall
# Add execution permission
chmod +x dht11_rdk_x5_gpio401_server
# Test the DHT11, then stop with Ctrl+C
sudo ./dht11_test
```

Figure 25 Build the helper programs and run the standalone DHT11 test.

Check that both gcc commands finish without an error. Keep the executable name dht11_rdk_x5_gpio401_server exactly as shown and place it beside line_follower.py. Both C programs use Linux GPIO 401 by default; if your verified wiring differs, update the GPIO definitions and paths in both C files before recompiling.

Test the DHT11 Before Integration

```
cd ~/RBE201TC
sudo ./dht11_test
```

Observe several valid temperature and relative humidity readings and the success/failure counts. The standalone test waits approximately two seconds between attempts. If readings fail, stop and check the supply, data wiring, GPIO configuration and competing programs. Press Ctrl+C to finish this test before starting line_follower.py.

Run a Short Integrated Test Route

```
cd ~/RBE201TC
date -Is
sudo python3 -c "import serial; import Hobot.GPIO; print('Dependencies OK')"
sudo python3 line_follower.py
```

The integrated application requires sudo. It starts dht11_rdk_x5_gpio401_server automatically with chrt -f 50 and taskset -c 7. Do not start the service separately or leave a standalone DHT11 test running. If startup fails, check the executable, GPIO sysfs interface, CPU 7 availability, scheduling permissions and dependencies before placing the robot on the floor.

Measurements are asynchronous: the robot continues moving after a transverse marker triggers a reading. A marker does not stop the robot or mark the end of the route. Record this behaviour when associating readings with station coordinates; a received result may represent air encountered after the trigger location.

1. Use a short, clearly marked black-tape route and add the transverse markers required by the program. Verify how the program handles markers before interpreting the data rows.

2. Use conservative settings to observe steering and stopping. Check that the expected temperature and humidity record is generated at each target marker.

3. Read the RunID and CSV path printed at startup. After stopping the run, open that file and check the columns, one row for each detected marker, final Status values and any blank measurements. Section 12 explains the file name and statuses. Save the code version and settings used.

4. Repeat the short test until the results are reproducible. Record failures and changes as well as successful runs.

Fault response: if a motion or braking command cannot be sent, or a CSV update cannot be saved, the integrated application stops its control loop and attempts to send stop commands. This is not physical confirmation that the wheels have stopped. If motion continues or communication is disconnected, immediately disconnect motor power. Resolve the fault and repeat the raised-wheel check before restarting.

Test item	Evidence
Line following	Short route diagram and run log
Temperature and humidity logging	Readable output file with units
Marker-to-record correspondence	Actual marker sequence and matching data rows
Reliable stopping and repeat runs	Test notes and known limitations

Checkpoint Completing the test route is an intermediate milestone. Next, measure the whole room and analyse the measured data.

12 Collect and Save Room Data

The baseline project must collect and analyse temperature and relative humidity measurements across the whole room.

Three repeat measurement runs are recommended.

1. Run the checked robot along the route and associate marker events with measurement stations. Create a station/event table keyed by RunID and Detection point, linked to the unedited CSV. Detection point is an event number, not a coordinate; numbering starts again at 1 in each new run.

2. Use the same stabilisation and sampling procedure throughout the measurements and record the fact that this code samples while moving. Record missing or repeated marker events and interventions. Stop and correct unreliable motion before continuing. Check reading delays before assigning measurements to station locations.

3. Repeat the measurements as planned. Each program start creates a separate CSV with a new RunID; retain all raw outputs, including failed runs, and record the software version, settings and room conditions.

4. Back up the raw files before cleaning. Create a separate analysis CSV and record all data associations, exclusions and corrections. Do not fabricate observations for unmeasured locations.

Read the Run Log

Each run creates temperature_points_<RunID>.csv in the same directory as line_follower.py. The program prints the exact file path at startup. RunID contains a UTC start timestamp and a random suffix, so repeat runs have separate files. Files from earlier runs are retained.

The CSV is encoded as UTF-8 with a byte-order mark. Its columns, in order, are shown below. Trigger time and Result time include a time-zone offset; Result time records when Python finalises the event, rather than the physical instant at which the sensor measured the air.

CSV column	Meaning
RunID	Identifier shared by all events in this run.
Detection point	Sequential detected marker number within this run.
Trigger time	Time recorded when the marker event is created.
Result time	Time the event reaches a final status; blank while pending.
Humidity_%RH	Relative humidity in %RH; blank unless the result succeeds.
Temperature_°C	Temperature in degrees Celsius; blank unless the result succeeds.
Status	Event state listed in the next table.
Explanation	Failure details or other information about the result.

Each detected event is first saved as one pending row before a sensor request is sent. The same row is then updated to one final status; a pending row and its final result are not two separate measurements. Failed or unfinished readings remain blank, never zero.

Status	Interpretation
pending	Event saved; its result is not yet final. Do not use it as a measurement.
success	A valid sensor result was received and saved.
sensor_failure	The service reported a failed or invalid reading, or ended before returning a pending result.
request_failure	The service was unavailable when sending, or the request could not be delivered.
timeout	No final result arrived within the default 10-second limit.
cancelled	The application stopped before this event received a final result.

Sensor failures, request failures and timeouts are recorded while line following can continue. Review these events and investigate missing data before another run. If power is lost, the process is forcibly terminated, or saving the log fails, the last saved CSV may still contain pending rows. Treat these as unfinished events and retain the original file.

Export the Data After Stopping

Stop the program with Ctrl+C and verify that the wheels have stopped before copying the CSV. On the RDK, list the saved run files:

```
ls -lt ~/RBE201TC/temperature_points_*.csv
```

On your group computer, open a terminal in the folder where you want the backup. In the command below, replace RDK_IP with the current board address and RUNID with the exact identifier from the chosen file name:

```
scp sunrise@RDK_IP:~/RBE201TC/temperature_points_RUNID.csv .
```

Open the copied file and check the RunID, header and rows against the RDK copy. Keep every run CSV unchanged with its station/event table, source version and settings. Create cleaned data in a separate file and use only success rows as valid measurements; retain and explain all other statuses.

Recommended Analysis Data Fields

Field	Meaning
GroupID, RoomID, RunID	Traceable group and room identifiers, and the RunID from the raw CSV
StationID	Unique station identifier matching the floor plan
Timestamp	Date and time in ISO-style format, including the time zone
X_m, Y_m	Station coordinates measured manually or with instruments, in metres
SensorHeight_m	Height above the floor, in metres
Temperature_C	Measured temperature, in degrees Celsius
RH_pct	Measured relative humidity, as a percentage
Status, Notes	Retain the raw event status; only success is a valid reading. Record any separate analysis flags or interventions.

Manually measured coordinates are acceptable, but record the origin and measurement method. Keep a separate log of battery status, room conditions, timing and equipment changes.

Checkpoint Check coverage across the room, sample counts, timestamps and error flags. Each observation must be traceable to a station, run and raw file. Explain gaps in coverage and investigate outliers using the sensor specifications and room conditions.

13 Troubleshoot and Return the Kit

Symptom	Checks in order
RDK does not start or repeatedly restarts	Stop the motors. Check the SD card, regulated supply, connectors and power capacity. Do not power the RDK from a computer USB port.
SSH connection fails	Check the active network, current address, access within the same subnet and account details. Reconnect using the new address after a static IP change.

Symptom	Checks in order
Serial port missing or access denied	Check the data cable, device path, dialout group permissions after logging back in, and whether another program is using the port.
Motor rotates in the wrong direction	Stop and disconnect motor power. Check the channel mapping and configuration.
Line sensor input does not change	Check wiring, height, surface contrast, signal polarity and input mapping. Repeat the individual channel test.
No temperature or humidity readings or helper program fails	Check sudo, dependency imports, the compiled executable, VCC/GND/OUT, GPIO 401, CPU 7 and scheduling permissions. Stop standalone tests before integration. The application starts its helper automatically.
Station records missing or duplicated	Compare the actual marker sequence with RunID and Detection point in that run CSV. Check Status and Explanation. Investigate missed/repeated triggers and duplicate imports; preserve failed or unfinished rows.
Room measurements are inconsistent	Before excluding data, check sensor position, stabilisation time, clock drift, run conditions, coordinates and uncertainty.
Motion or braking command fails	The application attempts to stop and exit. Check that the wheels stopped; disconnect motor power if needed. With power off, check the USB cable, serial port and driver before another raised-wheel test.
CSV cannot be saved or pending remains after exit	The application stops on a save error. Check disk space, filesystem access and the printed error. Preserve the last saved file; pending rows are unfinished events after a save failure, power loss or forced termination.

End the Laboratory Session Safely

1. Stop the application, confirm that all wheels have stopped, and disconnect motor power.

2. Copy every run CSV, the station/event table, configuration and source code version to the group archive. Confirm that the copied files open correctly and keep raw CSV files unchanged.

3. Shut down the RDK normally before disconnecting power. Do not remove the SD card while the operating system is running.